Self-Evolving Matrix-Poison Agent on Recommender System

Anonymous ACL submission

Abstract

Recommender systems are increasingly vulnerable to poisoning attacks, where malicious user data is injected to bias collaborative filtering models. Existing methods typically focus on single-item promotion on individual platforms, overlooking category-level attacks that exploit coordinated multi-account operations to amplify influence and enhance stealth. To address this, we propose Matrix-Poison, a self-evolving LLM-driven agent framework inspired by real-world Matrix Account Advertising. Matrix-Poison overcomes four key hurdles: vulnerability discovery, adaptive black-box planning, autonomous account synergism, and self-correcting stealth. Through iterative reconnaissance and strategy refinement, the agent balances attack impact with concealment. Experiments on real-world datasets demonstrate that Matrix-Poison significantly outperforms state-of-the-art baselines and exhibits strong cross-market portability. Code is available at: https://anonymous.4open.science/r/T83EC/.

1 Introduction

Recommender systems are foundational to modern e-commerce and media platforms because they shape what users see and consume. However, their reliance on user–item interaction data also makes them susceptible to poisoning attacks, in which adversaries inject malicious interactions to distort ranking outcomes (Wang et al., 2024b; Nguyen et al., 2024). Most prior studies focus on item-level manipulation, including fake-user injection, graph perturbation, and adversarial profile generation (Yin, 2024; Cheng, 2024; Zhang et al., 2023). Although effective, these methods are typically tailored to single-item, single-platform settings, which limits their applicability to broader, coordinated real-world manipulation campaigns.

In practice, adversarial promotion and suppression are often coordinated at scale. Matrix Account Advertising (MAA) uses multiple controlled accounts to shape user perception across scenarios (He et al., 2020a; Rancati and Gordini, 2014). Motivated by this pattern, we investigate *category-level poisoning*, where attackers manipulate a target category rather than a single item. This setting naturally supports both *push* and *nuke* objectives (Lam and Riedl, 2004): push attacks increase exposure of category-related targets, while nuke attacks suppress their visibility by steering collaborative signals away from them. Compared with item-level attacks, category-level manipulation can induce broader influence while remaining more stealthy through dispersed interaction footprints.

Despite its practical relevance, category-level poisoning in modern recommenders remains underexplored because it requires adaptive decision-making in gray-box environments. We identify four coupled challenges: (1) **Vulnerability Discovery**: locating category-level weak points beyond single-item sensitivity; (2) **Adaptive Planning**: adjusting attack distributions without access to model internals; (3) **Autonomous Synergism**: coordinating multi-account behaviors to amplify effect; and (4) **Self-Correcting Stealth**: continuously balancing attack impact and detectability. Static poisoning strategies struggle to satisfy all four requirements simultaneously.

To address these challenges, we propose **Matrix-Poison**, a self-evolving LLM-driven agent framework for category-level recommender poisoning. Matrix-Poison unifies analysis, planning, and judgment in a closed loop: it profiles scenario vulnerabilities, generates adaptive multi-account attack plans, and iteratively refines them using feedback from rapid validation. This design enables effective manipulation under gray-box constraints while preserving stealth through distribution-aware control. Extensive experiments on multiple real-world datasets and cross-market settings show that Matrix-Poison consistently outperforms strong

baselines for both push and nuke objectives, while maintaining high concealment and transferability. Our main contributions are as follows:

- We formulate category-level poisoning as a practical threat model for gray-box recommender systems, covering both push and nuke objectives.
- We introduce Matrix-Poison, a self-evolving LLM-agent framework that unifies vulnerability analysis, adaptive multi-account planning, and feedback-driven refinement.
- We conduct comprehensive experiments in single-market and cross-market settings, demonstrating stronger attack effectiveness than competitive baselines while offering favorable concealment and transferability.

2 Related Work

2.1 Poisoning Attacks on Recommender Systems

Poisoning attacks manipulate recommendation outcomes by injecting malicious interactions into training data (Aktukmak et al., 2019; Chung et al., 2013). Early studies mainly rely on hand-crafted attack profiles and heuristic injection strategies, whereas more recent methods use learned generators (e.g., GAN or RL-based) to improve attack strength and transferability (Aggarwal et al., 2021; Wang et al., 2024b; Nguyen, 2024). In parallel, prior work has examined vulnerabilities in collaborative filtering and graph-based recommenders (Aditya et al., 2016; Aggarwal et al., 2016; Das et al., 2017; Sachan and Richariya, 2013; Zhang et al., 2020), and has discussed broader robustness-related concerns such as distribution shift and fairness (Cai and Zhang, 2019; Abdollahi and Nasraoui, 2018). However, most existing poisoning methods remain item-level and largely static, leaving adaptive category-level poisoning insufficiently studied.

2.2 LLM-Empowered Agent Systems

LLM-empowered agent systems have recently improved recommendation and retrieval pipelines through task decomposition, planning, and tool use (Zhang et al., 2024a). Representative works explore multi-agent orchestration, interaction modeling, and workflow-level control (Rosenberg et al., 2021; Fang et al., 2024; Cai et al., 2024; Nie et al., 2024). Other studies further strengthen practical agent capabilities via memory, tool invocation, and feedback integration (Wang et al., 2024a; Hadi et al., 2023; Zhang et al., 2024b; Wang et al., 2023b; Shu et al., 2024; Zhao et al., 2024). Despite these advances, the primary objective remains recommendation utility under benign settings; explicit optimization of poisoning effectiveness—stealth trade-offs in gray-box environments remains limited (Ning et al., 2024).

2.3 Self-Evolving Agents

Self-evolving agents focus on iterative self-improvement through reflection, memory, and feedback. Prior work includes iterative refinement and critique-based correction (Madaan et al., 2023; Gou et al., 2023; Wang et al., 2023c), embodied lifelong learning (Wang et al., 2023a), evolutionary multi-agent optimization (Yuan et al., 2025; Zhai et al., 2025; Hu et al., 2024), and scalable memory/reasoning frameworks (Wang et al., 2025a; Cao et al., 2025; Ouyang et al., 2025). Recent systems also report strong agentic performance in coding and production-style environments (Zeng et al., 2025; Shang et al., 2025; Wang et al., 2025b; Xia et al., 2025). These results suggest that self-evolving paradigms are promising for long-horizon closed-loop optimization, motivating our adaptation to category-level poisoning in recommender systems.

3 Preliminaries

Recommendation Task. Let $\mathcal{U}$ and $\mathcal{I}$ denote the sets of users and items. For each user $u \in \mathcal{U}$, the historical interactions are represented as a chronological sequence

$$s_u = (i_1^{(u)}, \ldots, i_{L_u}^{(u)}), \tag{1}$$

where $i_t^{(u)} \in \mathcal{I}$ is the item interacted with at step t. General recommenders use the interaction set $\mathcal{H}_u = \{v \in s_u\}$, while sequential recommenders directly model s_u. The victim model predicts the preference score of user u on item v as

$$\hat{y}_{u,v} = f_\theta(u, v, \mathcal{C}_u), \tag{2}$$

where $\mathcal{C}_u = s_u$ for sequential models and $\mathcal{C}_u = \mathcal{H}_u$ for general models.

Category-level Poisoning Attack. Given a target category $\mathcal{C}_t \subseteq \mathcal{I}$ and a target item $i_t \in \mathcal{C}_t$, the attacker injects malicious users $\mathcal{U}_M$ with fabricated interactions $\mathcal{D}_M$ into the clean dataset $\mathcal{D}$, yielding

$$\mathcal{D}' = \mathcal{D} \cup \mathcal{D}_M. \tag{3}$$

We measure the attack effect by the prediction shift of the target item:

$$\delta_p(i_t) = \mathbb{E}_{u \in \mathcal{U}}\big[f_{\theta'}(u, i_t, \mathcal{C}_u) - f_\theta(u, i_t, \mathcal{C}_u)\big], \quad (4)$$

where $f_{\theta'}$ and f_θ are trained on $\mathcal{D}'$ and $\mathcal{D}$, respectively. A positive $\delta_p(i_t)$ indicates a push attack, and a negative one indicates a nuke attack.

Threat Model and Objective. We consider a gray-box attacker who has no access to the victim model parameters or gradients, but can collect a shadow dataset $\mathcal{D}_{\text{shadow}}$ and train a surrogate recommender f_{shadow}. Given a target category $\mathcal{C}_t$ and a target item $i_t \in \mathcal{C}_t$, the attacker seeks a malicious interaction set $\mathcal{D}_M$ that maximizes the attack objective under poisoning-budget and stealth constraints:

$$\begin{aligned} \max_{\mathcal{D}_M} \mathcal{O}_{\text{adv}}(\mathcal{D}_M) \quad \text{s.t.} \quad |\mathcal{U}_M| = \text{round}(|\mathcal{U}|\epsilon), \\ S(\mathcal{D}, \mathcal{D}') \le \epsilon_s,\ \mathcal{D}' = \mathcal{D} \cup \mathcal{D}_M. \end{aligned} \quad (5)$$

where $\mathcal{O}_{\text{adv}}$ is instantiated as the push or nuke objective in Section 4.3, ϵ is the poisoning budget, and ϵ_s is the maximum allowed stealth penalty.

4 Methodology

Matrix-Poison in Figure 1 iteratively performs initialization, analysis, planning, judging, and self-evolution for category-level poisoning.

4.1 Initialization and Heuristic Attacks

The framework first profiles the target category $\mathcal{C}_t$. Given dataset $\mathcal{D}$, it computes the category rating mean and variance:

$$\mu_{\mathcal{C}_t} = \frac{1}{|\mathcal{C}_t|}\sum_{i \in \mathcal{C}_t} \mu_i, \quad \sigma^2_{\mathcal{C}_t} = \frac{1}{|\mathcal{C}_t|}\sum_{i \in \mathcal{C}_t} (\mu_i - \mu_{\mathcal{C}_t})^2, \quad (6)$$

where μ_i is the average rating of item i. We use a unified parameterized heuristic generator

$$\mathcal{D}_M^{(t)} = \mathcal{M}(\boldsymbol{\theta}_t; \mathcal{D}, \mathcal{C}_t), \quad \boldsymbol{\theta}_t = (\epsilon_t, \mathbf{w}_t, \boldsymbol{\eta}_t), \quad \mathbf{w}_t \in \Delta^{K-1}, \quad (7)$$

where ϵ_t is the poisoning ratio, $\mathbf{w}_t$ controls mixture proportions over an internal atomic-attack basis, $\boldsymbol{\eta}_t$ denotes auxiliary generation parameters, K is the number of atomic attack components, and Δ^{K-1} denotes the probability simplex. The initialization profile specifies the attack mode (`Push` or `Nuke`), ϵ_0, and initial generator parameters. The number of injected malicious users is $|\mathcal{U}_M^{(t)}| = \text{round}(|\mathcal{U}|\epsilon_t)$.

4.2 Analysis Phase

The analysis phase estimates target-category vulnerability by lightweight probing on a shadow recommender. For each candidate non-budget configuration $\boldsymbol{\phi}_j = (\mathbf{w}_j, \boldsymbol{\eta}_j)$, it forms a temporary probe setting with

$$\epsilon_{\text{probe}} = \max(\epsilon_{\min}, \alpha_{\text{probe}}\epsilon_t), \quad (8)$$

where α_{probe} is the probe coefficient and $\epsilon_{\min}$ is the minimum probing ratio. Probe samples are used only for sensitivity estimation and are not included in the submitted poisoning set.

After injecting each probe set, the shadow recommender is briefly retrained. The category-level prediction shift is

$$\Delta_p(\boldsymbol{\phi}_j) = \frac{1}{|\mathcal{U}_e||\mathcal{C}_t|}\sum_{u \in \mathcal{U}_e}\sum_{i \in \mathcal{C}_t}\left[f_{\text{poison}}(u, i) - f_{\text{clean}}(u, i)\right], \quad (9)$$

where $\mathcal{U}_e$ denotes evaluation users. These scores rank candidate settings into $\mathcal{S}_{rank}$. The LLM analysis agent returns this ranking as strict JSON; invalid outputs fall back to deterministic score-based ranking.

4.3 Planning and Judging Phase

Given $\mathcal{S}_{rank}$ over candidates $\{\boldsymbol{\phi}_j\}_{j=1}^J$, the planner builds rank-aware weights:

$$\alpha_{t,j} \ge 0, \quad \sum_{j=1}^J \alpha_{t,j} = 1, \quad \tilde{\alpha}_{t,j} = \frac{1/r_j}{\sum_{\ell=1}^J 1/r_\ell}, \quad (10)$$

where r_j is the rank of candidate $\boldsymbol{\phi}_j$. Here, $\boldsymbol{\alpha}_t = \{\alpha_{t,j}\}_{j=1}^J$ denotes the proposal distribution over candidate settings, and $\tilde{\alpha}_{t,j}$ is its rank-based default initialization. The candidates are aggregated into $\boldsymbol{\theta}_t = (\epsilon_t, \mathbf{w}_t, \boldsymbol{\eta}_t)$, with mixture components projected to the simplex. The system then generates

$$\mathcal{D}_M^{(t)} = \mathcal{M}(\boldsymbol{\theta}_t; \mathcal{D}, \mathcal{C}_t), \quad \mathcal{D}' = \mathcal{D} \cup \mathcal{D}_M^{(t)}. \quad (11)$$

We use $\Delta_p^{(t)}$ to denote the realized prediction shift of $\mathcal{D}_M^{(t)}$ at iteration t.

The Judging Phase evaluates effectiveness and stealth. For push attacks:

$$\begin{aligned} \mathcal{O}_t^{\text{push}} = w_s\Delta_p^{(t)} + w_r(\Delta\text{Hit@}k_t + \Delta\text{NDCG@}k_t) \\ - \eta_1[\tau_p - \Delta_p^{(t)}]_+ - \eta_2[\Delta_{\text{clean},t} - \tau_c]_+. \end{aligned} \quad (12)$$

For nuke attacks:

$$\begin{aligned} \mathcal{O}_t^{\text{nuke}} = w_s(-\Delta_p^{(t)}) + w_r(-\Delta\text{Hit@}k_t - \Delta\text{NDCG@}k_t) \\ - \eta_3[\Delta_p^{(t)}]_+. \end{aligned} \quad (13)$$

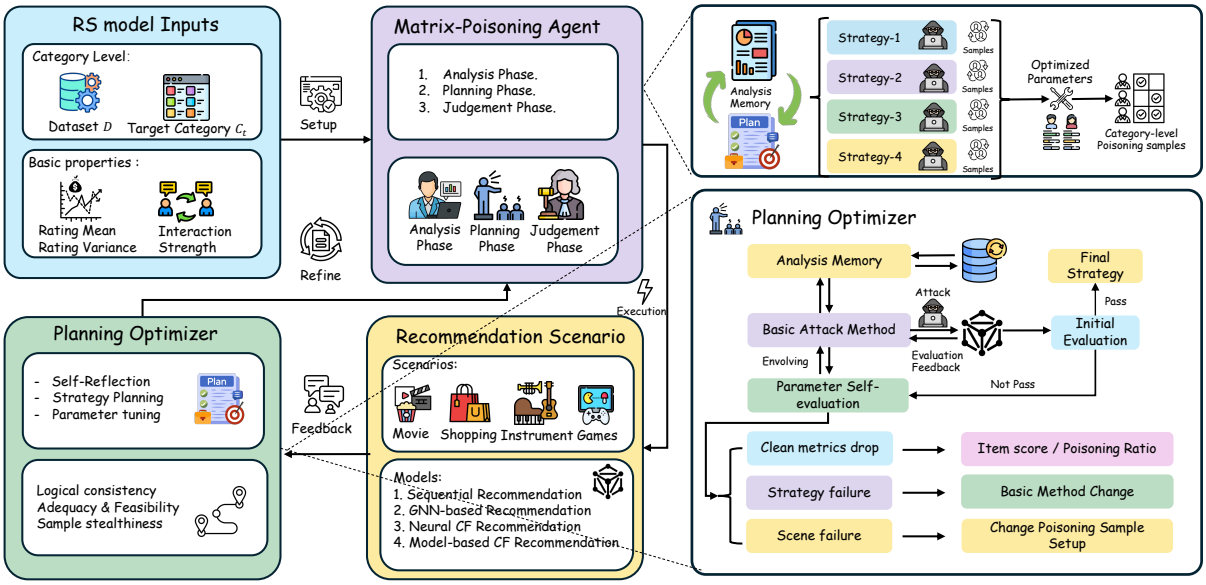

Figure 1: Overview of the Matrix-Poison Agent. The RS model input is responsible for loading the dataset and target category; the Matrix-Poison Agent is responsible for analyzing shortcomings in the recommendation scenario and formulating preliminary attack strategies; the Planning Optimizer optimizes the poisoning strategy through Self-evolving.

where k_t denotes the evaluation cutoff at iteration t, $\Delta_{\text{clean},t}$ denotes the utility drop on non-target clean evaluation data, $w_s, w_r, \eta_1, \eta_2, \eta_3$ are weighting coefficients, and τ_p, τ_c are feasibility thresholds. The reward is

$$R_t = \mathcal{O}_t - \lambda_s S(\mathcal{D}, \mathcal{D}'), \tag{14}$$

where λ_s controls the stealth penalty.

Stealth Formalization. Stealth combines rating shift, injected-profile length deviation, and interaction expansion. With fixed-bin rating histograms $h^r_{\mathcal{D}}$ and $h^r_{\mathcal{D}'}$,

$$S_{\text{rate}} = D_{\text{KL}}\left(h^r_{\mathcal{D}} \| h^r_{\mathcal{D}'}\right). \tag{15}$$

Let $n_{\mathcal{D}}(u)$ be the profile length of user u, and $n_{\mathcal{D}_M}(u)$ that of injected user $u \in \mathcal{U}_M^{(t)}$. We define

$$S_{\text{prof}} = \frac{\left| \frac{1}{|\mathcal{U}_M^{(t)}|} \sum_{u \in \mathcal{U}_M^{(t)}} n_{\mathcal{D}_M}(u) - \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} n_{\mathcal{D}}(u) \right|}{\frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} n_{\mathcal{D}}(u)}. \tag{16}$$

With $\rho_{\text{int}} = |\mathcal{D}'|/|\mathcal{D}|$, the stealth penalty is

$$S(\mathcal{D}, \mathcal{D}') = S_{\text{rate}} + S_{\text{prof}} + \gamma[\rho_{\text{int}} - 1]_+, \tag{17}$$

where γ is the interaction-expansion coefficient. Candidates are accepted only when they satisfy target-shift, clean-utility, interaction-ratio, and stealth-threshold constraints.

4.4 Self-evolving Mechanism Formalization

Self-evolution updates $\boldsymbol{\theta}_t = (\epsilon_t, \mathbf{w}_t, \boldsymbol{\eta}_t)$ using measurable feedback rather than free-form textual edits. At iteration t, the recorded state is

$$M_t = \{\boldsymbol{\theta}_t, \mathcal{O}_t, R_t, \Delta_p^{(t)}, \Delta\text{Hit@}k_t, \Delta\text{NDCG@}k_t, S_t, \Delta_{\text{clean},t}\}, \tag{18}$$

where $S_t = S(\mathcal{D}, \mathcal{D}')$. The parsed feedback is

$$g_t = \text{Parse}(M_t) = \langle d_t, a_t, q_t \rangle, \tag{19}$$

where d_t is the update direction, a_t the action type, and q_t the quantitative trigger.

The poisoning ratio is updated by

$$\epsilon_{t+1} = \text{clip}(\epsilon_t \kappa_t, \epsilon_{\min}, \epsilon_{\max}^{\text{mode}}), \tag{20}$$

where $\epsilon_{\max}^{\text{mode}}$ is the mode-specific upper bound of the poisoning ratio, and κ_t is selected from threshold rules over S_t, $\Delta_p^{(t)}$, and $\mathcal{O}_t$.

The remaining parameters are updated by memory blending and stagnation exploration:

$$\begin{aligned} \boldsymbol{\theta}_{t+1} &\leftarrow \text{Blend}(\boldsymbol{\theta}_t, \boldsymbol{\theta}_{\text{eff}}; \lambda_e), \\ \boldsymbol{\theta}_{t+1} &\leftarrow \text{Blend}(\boldsymbol{\theta}_{t+1}, \boldsymbol{\theta}_{\text{stealth}}; \lambda_m). \end{aligned} \tag{21}$$

where $\boldsymbol{\theta}_{\text{eff}}$ and $\boldsymbol{\theta}_{\text{stealth}}$ are the best effective and best stealthy historical settings, and λ_e, λ_m are memory blending factors. $\text{Blend}(\cdot)$ denotes parameter-wise interpolation followed by feasibility projection, and $\text{Parse}(\cdot)$ maps logged metrics to structured update feedback. If progress stagnates, a small mass in $\mathbf{w}_{t+1}$ is shifted from the dominant component to a runner-up component selected by the latest sensitivity ranking, followed by simplex projection. All updates are clipped to feasible bounds, and LLM suggestions are accepted only after JSON parsing and validation.

5 Experiments

5.1 Research Questions

In this section, we evaluate the proposed method on multiple real-world recommender system datasets and in both single-market and cross-market scenarios. The experiments are designed to answer the following research questions:

- **RQ1:** How effective is the Matrix-Poison framework in performing category-level poisoning attacks on recommender systems?
- **RQ2:** How effective is the Matrix-Poison framework in performing category-level cross-market poisoning attacks on recommender systems?
- **RQ3:** How well does the Matrix-Poison framework preserve stealthiness during poisoning attacks?

5.2 Experiment Settings

5.2.1 Datasets

Following previous work (Zhao et al., 2025; Wu et al., 2021), we conduct experiments on **ML-1M** (Harper and Konstan, 2015), **LastFM** (Zhu et al., 2018), and **Amazon Review** (Hou et al., 2024), which correspond to several recommendation scenarios. We further use **XMarket** (Bonab et al., 2021) to evaluate cross-market poisoning attacks. Dataset details shown in Appendix A.1.

5.2.2 Victim Recommender Systems

We evaluate Matrix-Poison on representative recommender systems: **BPR** (Rendle et al., 2012), **NeuMF** (He et al., 2017), **LightGCN** (He et al., 2020b), and **SASRec** (Kang and McAuley, 2018). These models cover matrix-factorization, neural collaborative filtering, graph-based recommendation, and sequential recommendation settings. Details of the victim models shown in Appendix A.2.

5.2.3 Implementation Details

We evaluate BPR, LightGCN, NeuMF, and SASRec on the ML-1M and LastFM datasets. All models are trained for 50 epochs with a learning rate of 0.001 under an 80:10:10 train/validation/test split. We report Hit@10, NDCG@10, and Prediction Shift. NeuMF uses cross-entropy loss, while SASRec uses a maximum sequence length of 50. The Matrix-Poison Agent is powered by DeepSeek-R1 and GPT-4o, with at most five self-correction iterations. The initial poisoning ratio is set to $\epsilon_0 = 0.005$, and the self-evolving optimizer updates ϵ_t within the predefined budget range when adaptive search is enabled. For controlled baseline comparisons, all methods are initialized with the same ϵ_0 and random seed 42, 54, 46. The experimental results are the averages of trials performed with multiple random number seeds.

5.2.4 Evaluation Metrics

Following prior work (Fang et al., 2018), we report Hit@k, NDCG@k, and Prediction Shift (P. Shift) as the main evaluation metrics. Since Matrix-Poison studies category-level manipulation, we explicitly evaluate exposure with respect to the target category C_t. For each evaluation user u, the recommender produces a top-k list $\mathcal{R}_k(u)$ after filtering items observed in the training set. Category-level Hit@k is defined as whether at least one item from C_t appears in $\mathcal{R}_k(u)$, while category-level NDCG@k measures the ranking position of target-category items within the same list. Prediction Shift measures the change in the predicted score of the selected target item $i_t \in C_t$ before and after poisoning. The detailed formulas are provided in Appendix A.4.

5.3 Single Market Attack Effectiveness

To address **RQ1**, we evaluate the effectiveness of Matrix-Poison for targeted poisoning in single-market recommender systems. The experiments are conducted in a gray-box setting, where the attacker has no access to the victim model's internal parameters. We use multiple datasets and victim recommenders to examine whether Matrix-Poison can generalize across different recommendation scenarios.

5.3.1 Targeted Attack on Recommender System via Matrix-Poison

To assess the impact of Matrix-Poison on targeted recommender systems, we inject synthetic user profiles into the training data and evaluate the resulting changes in target-category recommendation outcomes. We consider both **Nuke** attacks, which suppress target-category exposure, and **Push** attacks, which increase target-category exposure. During testing, the target category is randomly selected. Compared with conventional single-item poisoning attacks, Matrix-Poison operates at the category level and adaptively plans multi-account poisoning strategies.

The results in Table 2 show that Matrix-Poison manipulates category-level recommendation outcomes under both **Nuke** and **Push** settings. In

Methods	Models	ML-1M			LastFM			Automotive		
		Hit@10	NDCG@10	P. Shift	Hit@10	NDCG@10	P. Shift	Hit@10	NDCG@10	P. Shift
Baseline	BPR	0.2160	0.0966	-	0.2740	0.1249	-	0.0880	0.0458	-
	NeuMF	0.0920	0.0442	-	0.2600	0.1030	-	0.2470	0.2470	-
	LightGCN	0.2870	0.1311	-	0.4790	0.2087	-	0.2490	0.1612	-
	SASRec	0.5070	0.4117	-	0.0070	0.0027	-	0.2760	0.2110	-
AUSH (Lin et al., 2020)	BPR	0.1960	0.0800	-0.5892	0.1830	0.0699	-2.4266	0.0080	0.0040	-0.0031
	NeuMF	0.2540	0.1188	0.2002	0.8400	0.3225	-0.3345	0.2560	0.1993	-1.5154
	LightGCN	0.3150	0.1136	0.0543	0.6250	0.2373	-3.2556	0.1640	0.0918	-0.0228
	SASRec	0.6130	0.4494	-0.1804	0.0000	0.0000	-1.9390	0.5960	0.2094	0.2080
GOAT (Wu et al., 2021)	BPR	0.1650	0.0613	-0.5548	0.1630	0.0702	-2.4438	0.0130	0.0068	-0.0035
	NeuMF	0.2400	0.1272	0.3444	0.7890	0.3889	-0.5069	0.1140	0.1097	-1.6649
	LightGCN	0.3160	0.1143	0.0697	0.6520	0.2666	-3.2893	0.1900	0.1204	-0.0232
	SASRec	0.6230	0.5571	-0.0159	0.0000	0.0000	-1.9709	0.9310	0.4610	0.2486
PC-Attack (Zeng et al., 2023)	BPR	0.1500	0.0568	-0.6018	0.2270	0.1063	-2.4062	0.0110	0.0058	-0.0034
	NeuMF	0.1450	0.0547	-0.0819	0.7890	0.3541	-0.5298	0.2470	0.1278	-1.5374
	LightGCN	0.2320	0.0831	-0.0147	0.5610	0.2232	-3.3354	0.2180	0.1448	-0.0231
	SASRec	0.4790	0.1843	-0.3750	0.0000	0.0000	-2.0066	0.0550	0.0177	0.2757
Matrix Poison-Nuke	BPR	0.0810	0.0302	-0.7241	0.1330	0.0546	-2.5712	0.0180	0.0082	-0.0058
	NeuMF	0.1530	0.0658	-0.5408	0.2670	0.0850	-0.9275	0.0000	0.0000	-1.7076
	LightGCN	0.0390	0.0140	-0.4911	0.4740	0.1926	-3.4254	0.2640	0.1584	-0.0296
	SASRec	0.2650	0.0852	-0.3966	0.0650	0.0240	-2.1739	0.0020	0.0006	-0.4660

Table 1: Comprehensive Performance Metrics with SOTA Methods.

the Nuke scenario, it substantially reduces target-category visibility for most victim models; for example, on ML-1M, Hit@10 drops from 0.2160 to 0.0810 for BPR and from 0.2870 to 0.0390 for LightGCN. In the Push scenario, it generally increases target-category exposure, especially on ML-1M, where all four models show clear gains in Hit@10 and NDCG@10. The effects are not uniform across all settings: some LightGCN results on LastFM and Automotive show weaker or unstable changes, suggesting model- and dataset-dependent robustness. Overall, Matrix-Poison adapts to different attack objectives while exposing heterogeneous vulnerabilities across recommenders.

5.3.2 Comparison with State-of-the-Art Methods

To compare Matrix-Poison with state-of-the-art recommender system poisoning methods, we evaluate it against baseline attacks, including Random, Average, Bandwagon, and Segmented attacks, as well as advanced AI-based approaches, including AUSH (Lin et al., 2020), GOAT (Wu et al., 2021), and PC-Attack (Zeng et al., 2023). The detailed target model settings are shown in section A.8. The comparison is conducted on the introduced datasets using Hit@10, NDCG@10, and Prediction Shift.

Table 1 compares Matrix-Poison with representative poisoning baselines under the Nuke setting. Matrix-Poison achieves stronger suppression on most model-dataset combinations: it obtains the lowest Hit@10 and NDCG@10 for BPR, LightGCN, and SASRec on ML-1M, strongly degrades BPR and SASRec on LastFM, and reduces target visibility to nearly zero for NeuMF and SASRec on Automotive. However, it does not dominate every case; for example, several baselines outperform it on Automotive-LightGCN. This suggests that attack effectiveness depends on dataset sparsity, model architecture, and category structure. Overall, Matrix-Poison provides the most consistent suppression across heterogeneous settings, supporting the value of adaptive multi-agent planning over fixed heuristic strategies.

5.4 Cross-market Poisoning Attack Effectiveness

We conduct cross-market experiments on the XMRec Amazon *Musical Instruments* subset across the US, JP, and UK markets. Categories are aligned by the same product-domain label, while user and item indices are rebuilt independently in each market. The markets differ in scale (US: 237,430 users/4,653 items; JP: 17,992 users/1,217 items; UK: 77,274 users/2,296 items) and only partially overlap in products, with 1,067 items shared by all three. To avoid target-market leakage, we do not transfer fake users, generated profiles, model parameters, interaction records, or market-specific IDs. Instead, only high-level attack decisions, including attack intent, poisoning ratio ϵ, profile-length constraint, and mixture policy, are transferred; malicious profiles are regenerated locally using the aligned category pool. All markets use the same poisoning ratio $|\mathcal{U}_M^m| = \mathrm{round}(|\mathcal{U}^m|\epsilon)$, victim model, split protocol, profile-length bound, cutoff, and metrics, with target items selected by the same popularity-based policy.

Methods	Models	ML-1M			LastFM			Automotive		
		Hit@10	NDCG@10	P. Shift	Hit@10	NDCG@10	P. Shift	Hit@10	NDCG@10	P. Shift
Baseline	BPR	0.2160	0.0966	-	0.2740	0.1249	-	0.0880	0.0458	-
	NeuMF	0.0920	0.0442	-	0.2600	0.1030	-	0.2470	0.2470	-
	LightGCN	0.2870	0.1311	-	0.4790	0.2087	-	0.2490	0.1612	-
	SASRec	0.5070	0.4117	-	0.0070	0.0027	-	0.2760	0.2110	-
Matrix Poison-Nuke	BPR	0.0810	0.0302	-0.7241	0.1330	0.0546	-2.5712	0.0180	0.0082	-0.0058
	NeuMF	0.1530	0.0658	-0.5408	0.2670	0.0850	-0.9275	0.0000	0.0000	-1.7076
	LightGCN	0.0390	0.0140	-0.4911	0.4740	0.1926	-3.4254	0.2640	0.1584	-0.0296
	SASRec	0.2650	0.0852	-0.3966	0.0650	0.0240	-2.1739	0.0020	0.0006	-0.4660
Matrix Poison-Push	BPR	0.6300	0.6281	0.5035	0.5821	0.2150	1.4502	0.0870	0.0532	0.0005
	NeuMF	0.5910	0.5352	1.1856	0.8820	0.8143	0.7130	0.6718	0.6430	0.0698
	LightGCN	0.6300	0.6300	2.2096	0.2390	0.1988	0.0004	0.6290	0.5628	0.0167
	SASRec	0.6310	0.6160	0.4869	0.8820	0.6817	0.6404	0.5810	0.6620	0.4055

Table 2: Comprehensive Performance Metrics with Baseline Methods. Best results are in bold.

Methods	Models	US			JP			UK		
		Hit@10	NDCG@10	P. Shift	Hit@10	NDCG@10	P. Shift	Hit@10	NDCG@10	P. Shift
Cross-market Baseline	BPR	0.0110	0.0054	-	0.4056	0.2478	-	0.4312	0.2720	-
	NeuMF	0.0076	0.0036	-	0.0440	0.0259	-	0.1282	0.0473	-
	LightGCN	0.0090	0.0054	-	0.4365	0.1984	-	0.3620	0.1333	-
	SASRec	0.0053	0.0022	-	0.3071	0.1044	-	0.1522	0.0594	-
Matrix Poison-Nuke	BPR	0.0020	0.0009	-0.0036	0.0056	0.0027	-0.0003	0.1520	0.0815	-0.0054
	NeuMF	0.0032	0.0014	-0.5836	0.0440	0.0224	-0.0206	0.0234	0.0169	-1.2579
	LightGCN	0.0019	0.0009	-0.0063	0.0297	0.0171	-0.0017	0.2532	0.1096	-0.0090
	SASRec	0.0018	0.0007	-0.2780	0.0390	0.1087	-0.0813	0.2358	0.0964	-0.8101
Matrix Poison-Push	BPR	0.0174	0.0097	0.0080	0.4582	0.2799	0.0382	0.4641	0.2949	0.6201
	NeuMF	0.1807	0.1674	1.2044	0.3050	0.2854	0.1759	0.6070	0.4540	0.7311
	LightGCN	0.0178	0.0096	0.0394	0.4390	0.2578	0.0085	0.4231	0.2116	0.6235
	SASRec	0.0137	0.0050	4.2201	0.4155	0.2187	0.0025	0.2129	0.1069	0.1809

Table 3: Comprehensive Performance Metrics with Baseline Methods in Cross-Market Scenarios.

Table 3 shows that Matrix-Poison can transfer category-level poisoning policies across markets, but the effect depends on the target region and victim model. In the U.S. market, Nuke consistently suppresses target exposure across all four recommenders, reducing Hit@10 to below 0.004. In Japan, it also produces clear degradation for BPR, LightGCN, and SASRec, suggesting that the learned policy can capture transferable attack patterns when markets share similar item spaces. The U.K. results are less uniform: NeuMF is strongly affected, whereas BPR and LightGCN show weaker degradation, and SASRec even increases in Hit@10. Push attacks similarly improve target visibility for BPR, NeuMF, and LightGCN, but remain less stable on SASRec. Overall, these results support the cross-market transferability of Matrix-Poison, while also showing that transfer is not guaranteed across all market–model combinations.

5.5 Stealthiness Evaluation on Matrix-Poison

To address **RQ3**, we evaluate Matrix-Poison's stealthiness from two perspectives: global system utility and statistical distribution similarity. First, we compare overall Hit@k and NDCG@k before and after poisoning to examine whether the injected profiles cause broad degradation to benign recommendation performance. Second, we analyze rating distributions and user embedding distributions to test whether malicious profiles remain statistically close to normal user behavior.

5.5.1 Impact on the Performance of Recommender Systems

In this section, we validate Matrix-Poison's stealthiness by evaluating Hit@k and NDCG@k before and after poisoning the victim recommender. We set $k = \{10, 50\}$ to account for both top-ranked and deeper recommendation lists. The results are shown in Fig. 2.

The results in Fig. 2 suggest that Matrix-Poison preserves the overall utility of the recommender system while manipulating the target category. Although the target category is strongly affected, the global Hit@k and NDCG@k metrics change only moderately after poisoning. The performance decrease is approximately 6% in the evaluated setting, indicating that the injected profiles primarily affect the selected target category rather than causing widespread degradation across the entire recommendation space. This supports the stealthiness

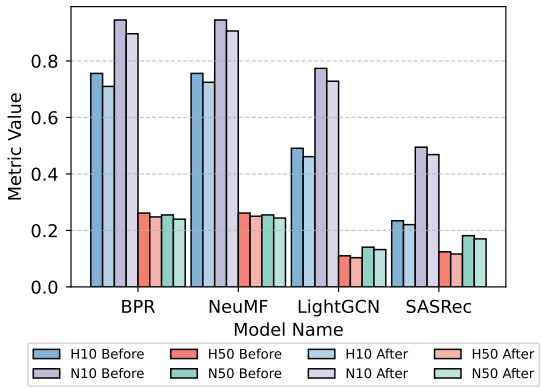

Figure 2: Performance Metrics Before and After Poisoning Attack.

claim from a system-utility perspective.

5.5.2 Sample Distribution Changes in Recommender Systems

We assess stealthiness by comparing pre- and post-poisoning sample distributions in observable behavior and latent user representations. For ML-1M, we use 1–5 ratings directly. For LastFM, playback counts are mapped into five interaction-intensity sections: `1--12`, `12--165`, `165--2130`, `2130--27410`, and `27410--352698` plays.

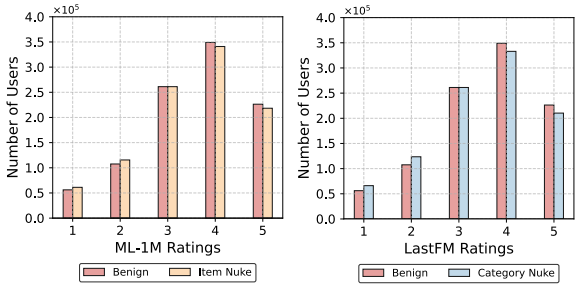

Figure 3: Rating/interaction-intensity distributions before and after poisoning on ML-1M and LastFM.

Fig. 3 shows that poisoning introduces only minor distributional changes. ML-1M remains concentrated on medium-to-high ratings, while LastFM preserves the same decreasing trend across interaction-intensity sections. The Kolmogorov-Smirnov test shows no significant shift ($p > 0.05$), and the average section-level change stays below 2%.

5.6 Ablation Study

To verify the effectiveness of the core components in Matrix-Poison, we conduct an ablation study on ML-1M under the Nuke attack setting. We compare the complete framework with four variants: **(1)** w/o Analysis Phase, which removes the vulnerability detection step; **(2)** w/o Planning Phase, which uses a single heuristic attack instead of adaptive strategy generation; **(3)** w/o Judgment Phase, which removes the self-reflection and optimization mechanism; and **(4)** w/o Self-Reflection Phase, which disables the self-evolving optimization process. The results are shown in Table 4.

Methods	Hit@10↓	NDCG@10↓	P. Shift↓
Full Matrix-Poison	**0.0810**	**0.0302**	**-1.3241**
w/o Analysis Phase	0.1850	0.0766	-1.1868
w/o Planning Phase	0.1990	0.0916	-1.1236
w/o Judgment Phase	0.1820	0.0782	-1.2024
w/o Self-Reflection Phase	0.2040	0.0884	-1.1418
w/o LLM-driven Attack	0.1982	0.0868	-0.8213

Table 4: Performance Metrics for Method Variants

The results in Table 4 show that each component contributes to attack performance. The full Matrix-Poison achieves the lowest Hit@10 and NDCG@10, while removing the Planning Phase causes the largest degradation, increasing Hit@10 from 0.0810 to 0.1990 and NDCG@10 from 0.0302 to 0.0916. Removing the Analysis or Judgment Phase also weakens the attack, indicating the importance of vulnerability profiling and iterative feedback. The w/o Self-Reflection variant performs worst among the ablated settings, further supporting the role of the self-evolving loop. At the same time, we remove the LLM-driven as the ablation condition and replace it with a fixed poisoning strategy, and the experimental results are significantly weaker than the Matrix-Poison method. Overall, the gains come from the combined analysis–planning–judgment–reflection pipeline rather than any single module.

6 Conclusion

We presented Matrix-Poison, a self-evolving LLM-driven framework for category-level poisoning attacks on recommender systems. By unifying analysis, planning, and judgment in a closed loop, the method adapts in gray-box settings, coordinates multi-account behaviors, and preserves stealth. Across datasets, models, and cross-market scenarios, Matrix-Poison outperforms existing baselines in both Push and Nuke attacks while maintaining high transferability. Our findings reveal practical vulnerabilities in collaborative filtering and motivate stronger defenses, including anomaly-aware detection, robust training, and poisoning-resistant data pipelines.

Limitations

This work has several limitations. First, MatrixPoison is evaluated in a gray-box setting that assumes access to a shadow dataset and surrogate training; its effectiveness in stricter black-box scenarios remains to be validated. Second, experiments are conducted on representative models and benchmark datasets, which may not fully capture the complexity of large-scale industrial deployments with stronger defenses and richer feedback dynamics. Third, the framework relies on powerful LLMs for planning and self-correction, thereby increasing computational cost and potentially reducing reproducibility across model providers and versions.

Ethics Statement

This work studies category-level poisoning of recommender systems in an offline gray-box setting. The purpose of the study is to evaluate system robustness and expose failure modes, not to facilitate deployment against live platforms. All experiments are conducted on public benchmark datasets, including ML-1M, LastFM, and Amazon Review/XMarket subsets, together with surrogate recommenders. No real user accounts, private logs, credential information, or external production APIs are accessed during the study.

Release Policy We will release benchmark construction scripts, preprocessing code, evaluation code, and sanitized analysis utilities to support reproducibility. However, we will not release end-to-end attack orchestration scripts, full agent trajectories, self-reflection traces, target-specific planning heuristics, or account-generation templates that could directly lower the barrier to misuse. Any additional artifact release will follow a research-only access policy and be reviewed for dual-use risk.

Attack Constraints All poisoning experiments are bound by a fixed budget and remain confined to offline simulation on public datasets. The attack does not target safety-critical domains, does not attempt persistence, credential abuse, rate-limit evasion, or real-time interaction with deployed recommender systems, and does not seek to bypass platform safeguards. The multi-account profiles used in the experiments are synthetic and exist only within the experimental environment.

Defensive Framing The central goal of MatrixPoison is to surface vulnerabilities in collaborative filtering and sequential recommendation under category-level poisoning, thereby informing anomaly detection, robust training, poisoning-resistant preprocessing, and auditing strategies. The reported results should therefore be interpreted as stress tests for defensive research, rather than as operational guidance for misuse.

Misuse Mitigation and Redaction Policy To reduce operational misuse, we redact platform-specific prompts, exact planning traces, and other implementation details that are unnecessary for understanding the scientific claims. The paper reports the attack pipeline at a conceptual level and preserves only the information needed to interpret the benchmark findings. We also present aggregate metrics rather than per-user or per-account traces, and we avoid exposing any content that would materially simplify direct replication on live systems.

Data Privacy and Consent All datasets used in this study are publicly available benchmark corpora. They do not contain direct personal identifiers, and the work does not involve human participants or live user studies. Formal consent procedures are therefore not applicable. Even so, we follow responsible data-use practices, apply filtering to remove sensitive or explicit content where relevant, and avoid releasing any unnecessary raw traces that could expose identifying patterns.

Transparency and Reproducibility We adhere to the ACL Responsible NLP Research checklist and document the experimental settings, random seeds, and evaluation metrics used in the paper. To balance transparency with safety, we provide sufficient detail for independent verification of the benchmark results and defense analysis, while withholding components whose unrestricted release would materially increase the risk of misuse.

References

Behnoush Abdollahi and Olfa Nasraoui. 2018. Transparency in fair machine learning: the case of explainable recommender systems. *Human and machine learning: Visible, explainable, trustworthy and transparent*.

PH Aditya, Indra Budi, and Qorib Munajat. 2016. A comparative analysis of memory-based and model-based collaborative filtering on the implementation of recommender system for e-commerce in indonesia:

A case study pt x. In *2016 International Conference on Advanced Computer Science and Information Systems (ICACSIS)*.

Alankrita Aggarwal, Mamta Mittal, and Gopi Battineni. 2021. Generative adversarial network: An overview of theory and applications. *International Journal of Information Management Data Insights*.

Charu C Aggarwal and 1 others. 2016. *Recommender systems*. Springer.

Mehmet Aktukmak, Yasin Yilmaz, and Ismail Uysal. 2019. Quick and accurate attack detection in recommender systems through user attributes. In *Proceedings of the 13th ACM Conference on Recommender Systems*.

Hamed Bonab, Mohammad Aliannejadi, Ali Vardasbi, Evangelos Kanoulas, and James Allan. 2021. Cross-market product recommendation. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. ACM.

Robin Burke, Bamshad Mobasher, and Runa Bhaumik. 2005. Limited knowledge shilling attacks in collaborative filtering systems. In *Proceedings of 3rd international workshop on intelligent techniques for web personalization (ITWP 2005), 19th international joint conference on artificial intelligence (IJCAI 2005)*.

Robin Burke, Bamshad Mobasher, Chad Williams, and Runa Bhaumik. 2006. Classification features for attack detection in collaborative recommender systems. In *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*.

Hongyun Cai and Fuzhi Zhang. 2019. Bs-sc: An unsupervised approach for detecting shilling profiles in collaborative recommender systems. *IEEE Transactions on Knowledge and Data Engineering*.

Shihao Cai, Jizhi Zhang, Keqin Bao, Chongming Gao, and Fuli Feng. 2024. Flow: A feedback loop framework for simultaneously enhancing recommendation and user agents. *arXiv preprint arXiv:2410.20027*.

Jiaqi Cao, Jiarui Wang, Rubin Wei, Qipeng Guo, Kai Chen, Bowen Zhou, and Zhouhan Lin. 2025. Memory decoder: A pretrained, plug-and-play memory for large language models. *arXiv preprint arXiv:2508.09874*.

Lei Cheng. 2024. Towards robust recommendation: A review and an adversarial robustness evaluation library. *arXiv preprint arXiv:2404.17844*.

Chen-Yao Chung, Ping-Yu Hsu, and Shih-Hsiang Huang. 2013. βp: A novel approach to filter out malicious rating profiles from recommender systems. *Decision Support Systems*.

Debashis Das, Laxman Sahoo, and Sujoy Datta. 2017. A survey on recommendation system. *International Journal of Computer Applications*, 160(7):6–10.

Jiabao Fang, Shen Gao, Pengjie Ren, Xiuying Chen, Suzan Verberne, and Zhaochun Ren. 2024. A multi-agent conversational recommender system. *arXiv preprint arXiv:2402.01135*.

Minghong Fang, Guolei Yang, Neil Zhenqiang Gong, and Jia Liu. 2018. Poisoning attacks to graph-based recommender systems. In *Proceedings of the 34th annual computer security applications conference*, pages 381–392.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2023. Critic: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738*.

Muhammad Usman Hadi, Rizwan Qureshi, Abbas Shah, Muhammad Irfan, Anas Zafar, Muhammad Bilal Shaikh, Naveed Akhtar, Jia Wu, Seyedali Mirjalili, and 1 others. 2023. A survey on large language models: Applications, challenges, limitations, and practical usage. *Authorea Preprints*.

F Maxwell Harper and Joseph A Konstan. 2015. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*.

Xiang He, Li Li, Hua Zhang, and Xingzhen Zhu. 2020a. Social media or website? research on online advertising type based on evolutionary game. In *Smart Business: Technology and Data Enabled Innovative Business Models and Practices: 18th Workshop on e-Business, WeB 2019, Munich, Germany, December 14, 2019, Revised Selected Papers 18*.

Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020b. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*.

Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*.

Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. 2024. Bridging language and items for retrieval and recommendation. *arXiv:2403.03952*.

Yue Hu, Yuzhu Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, Yuchen Hou, Shuo Tang, and Siheng Chen. 2024. Self-evolving multi-agent collaboration networks for software development. *arXiv preprint arXiv:2410.16946*.

Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*.

Shyong K Lam and John Riedl. 2004. Shilling recommender systems for fun and profit. In *Proceedings of the 13th international conference on World Wide Web*.

Chen Lin, Si Chen, Hui Li, Yanghua Xiao, Lianyun Li, and Qian Yang. 2020. Attacking recommender systems with augmented user profiles. In *Proceedings of the 29th ACM international conference on information & knowledge management*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594.

Thanh Tam Nguyen. 2024. Github - awesome recsys-poisoning.

Thanh Toan Nguyen, Nguyen Quoc Viet Hung, Thanh Tam Nguyen, Thanh Trung Huynh, Thanh Thi Nguyen, Matthias Weidlich, and Hongzhi Yin. 2024. Manipulating recommender systems: A survey of poisoning attacks and countermeasures. *ACM Computing Surveys*.

Guangtao Nie, Rong Zhi, Xiaofan Yan, Yufan Du, Xiangyang Zhang, Jianwei Chen, Mi Zhou, Hongshen Chen, Tianhao Li, Ziguang Cheng, and 1 others. 2024. A hybrid multi-agent conversational recommender system with llm and search engine in e-commerce. In *Proceedings of the 18th ACM Conference on Recommender Systems*.

Liang-bo Ning, Shijie Wang, Wenqi Fan, Qing Li, Xin Xu, Hao Chen, and Feiran Huang. 2024. Cheatagent: Attacking llm-empowered recommender systems via llm agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.

Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, and 1 others. 2025. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*.

Elisa Rancati and Niccolo Gordini. 2014. Content marketing metrics: Theoretical aspects and empirical evidence. *European Scientific Journal*.

Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2012. Bpr: Bayesian personalized ranking from implicit feedback. *arXiv preprint arXiv:1205.2618*.

Ishai Rosenberg, Asaf Shabtai, Yuval Elovici, and Lior Rokach. 2021. Adversarial machine learning attacks and defense methods in the cyber security domain. *ACM Computing Surveys (CSUR)*.

Atisha Sachan and Vineet Richariya. 2013. A survey on recommender systems based on collaborative filtering technique. *International journal of Innovations in Engineering and technology (IJIET)*.

Ning Shang, Yifei Liu, Yi Zhu, Li Lyna Zhang, Weijiang Xu, Xinyu Guan, Buze Zhang, Bingcheng Dong, Xudong Zhou, Bowen Zhang, and 1 others. 2025. rstar2-agent: Agentic reasoning technical report. *arXiv preprint arXiv:2508.20722*.

Yubo Shu, Haonan Zhang, Hansu Gu, Peng Zhang, Tun Lu, Dongsheng Li, and Ning Gu. 2024. Rah! recsys–assistant–human: A human-centered recommendation framework with llm agents. *IEEE Transactions on Computational Social Systems*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023a. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, and 1 others. 2024a. A survey on large language model based autonomous agents. *Frontiers of Computer Science*.

Piaohong Wang, Motong Tian, Jiaxian Li, Yuan Liang, Yuqing Wang, Qianben Chen, Tiannan Wang, Zhicong Lu, Jiawei Ma, Yuchen Eleanor Jiang, and 1 others. 2025a. O-mem: Omni memory system for personalized, long horizon, self-evolving agents. *arXiv e-prints*, pages arXiv–2511.

Xingyao Wang, Simon Rosenberg, Juan Michelini, Calvin Smith, Hoang Tran, Engel Nyst, Rohit Malhotra, Xuhui Zhou, Valerie Chen, Robert Brennan, and 1 others. 2025b. The openhands software agent sdk: A composable and extensible foundation for production agents. *arXiv preprint arXiv:2511.03690*.

Yancheng Wang, Ziyan Jiang, Zheng Chen, Fan Yang, Yingxue Zhou, Eunah Cho, Xing Fan, Xiaojiang Huang, Yanbin Lu, and Yingzhen Yang. 2023b. Recmind: Large language model powered agent for recommendation. *arXiv preprint arXiv:2308.14296*.

Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. 2023c. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents. *arXiv preprint arXiv:2302.01560*.

Zongwei Wang, Junliang Yu, Min Gao, Wei Yuan, Guanhua Ye, Shazia Sadiq, and Hongzhi Yin. 2024b. Poisoning attacks and defenses in recommender systems: A survey. *arXiv preprint arXiv:2406.01022*.

Fan Wu, Min Gao, Junliang Yu, Zongwei Wang, Kecheng Liu, and Xu Wang. 2021. Ready for emerging threats to recommender systems? a graph convolution-based generative shilling attack. *Information Sciences*.

Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. 2025. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint arXiv:2511.16043*.

Shuyuan Xu, Wenyue Hua, and Yongfeng Zhang. 2024. Openp5: An open-source platform for developing, training, and evaluating llm-based recommender systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.

Ming Yin. 2024. Poisoning federated recommender systems with fake users. *Preprint*, arXiv:2402.11637.

Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. 2025. Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6192–6217.

Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, and 1 others. 2025. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*.

Meifang Zeng, Ke Li, Bingchuan Jiang, Liujuan Cao, and Hui Li. 2023. Practical cross-system shilling attacks with limited access to data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 4864–4874.

Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, Shinji Mai, Li Yu, Jiaji Deng, Zouying Cao, and 1 others. 2025. Agentevolver: Towards efficient self-evolving agent system. *arXiv preprint arXiv:2511.10395*.

Chong Zhang, Xinyi Liu, Zhongmou Zhang, Mingyu Jin, Lingyao Li, Zhenting Wang, Wenyue Hua, Dong Shu, Suiyuan Zhu, Xiaobo Jin, and 1 others. 2024a. When ai meets finance (stockagent): Large language model-based stock trading in simulated real-world environments. *arXiv preprint arXiv:2407.18957*.

Junjie Zhang, Yupeng Hou, Ruobing Xie, Wenqi Sun, Julian McAuley, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2024b. Agentcf: Collaborative learning with autonomous language agents for recommender systems. In *Proceedings of the ACM Web Conference 2024*.

Peiyan Zhang, Jiayu Li, Chaozhuo Li, Xiang Li, Senzhang Wang, Jian Zhang, and Sunghun Kim. 2023. Poisoning gnn-based recommender systems with generative surrogate-based attacks. *ACM Transactions on Information Systems*.

Shijie Zhang, Hongzhi Yin, Tong Chen, Quoc Viet Nguyen Hung, Zi Huang, and Lizhen Cui. 2020. Gcn-based user representation learning for unifying robust recommendation and fraudster detection. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*.

Yuchuan Zhao, Tong Chen, Junliang Yu, Kai Zheng, Lizhen Cui, and Hongzhi Yin. 2025. Diversity-aware dual-promotion poisoning attack on sequential recommendation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1634–1644.

Yuyue Zhao, Jiancan Wu, Xiang Wang, Wei Tang, Dingxian Wang, and Maarten De Rijke. 2024. Let me do it for you: Towards llm empowered recommendation via tool learning. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.

Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning tree-based deep model for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*.

Appendix

A Additional Details

A.1 Dataset Details

This section provides a detailed description of the datasets used in our experiments, including their scale, domain characteristics, and preprocessing procedures.

- **ML-1M** (Harper and Konstan, 2015), from GroupLens Research, contains 1 million ratings from 6,040 users on 3,952 movies, including user demographics, movie genres, and timestamps, making it ideal for collaborative filtering and hybrid recommender systems.
- **LastFM** (Xu et al., 2024) captures listening histories, including 19 million records for 1,000 users in LastFM-1K, along with social and temporal data, which is used for music recommendation and graph-based models.
- **Amazon Review** (Hou et al., 2024) dataset is a large-scale Amazon product review dataset containing hundreds of millions of real-world reviews, ratings, textual feedback, and rich product metadata across a wide range of product categories. Benchmarking and pretraining for recommender systems, retrieval tasks, and multimodal models.
- **XMarket** (Bonab et al., 2021) XMarket is a project focused on Cross-Market Recommendation (XMRec), providing a dataset of recommender systems in the United States market (US) region. These datasets are organized by product category and include domains such as arts and crafts, automotive, books, and electronics.

Table 5: Statistics of datasets used in the evaluation of Matrix-Poison for recommender system attacks.

Dataset	#Users	#Items	#Interact.	Density
ML-1M	6,040	3,900	1,000,209	4.242%
LastFM	1,892	17,632	92,834	0.279%
Automotive	2,928	1,835	20,473	0.381%

We utilize three benchmark datasets (ML-1M, LastFM) and a specialized *Automotive* dataset for cross-market evaluation, with detailed statistics summarized in Table 5. To ensure fair comparison, we adopt a uniform implicit feedback setting across all models: ML-1M ratings of 4 or higher and LastFM play counts greater than 0 are treated as positive interactions. For the cross-market scenario, we select target items from niche categories to maximize attack visibility and leverage user overlap between the Musical Instruments dataset, which serves as the bridge for propagating poisoned signals.

A.2 Baseline Recommender Systems

The following Baseline Recommender Systems are used as our Target models, and their details are as follows:

- **BPR** (Rendle et al., 2012) optimizes user preference ranking for items by the Bayesian method, which is suitable for recommendation systems with implicit feedback.
- **NeuMF** (He et al., 2017) is a neural collaborative filtering algorithm that combines matrix factorization and a multilayer perceptron to capture the nonlinear characteristics of user-item interactions.
- **LightGCN** (He et al., 2020b) uses a lightweight graph convolutional network to capture the high-order relationships in the user-item interaction graph, simplify the model, and improve the recommendation performance.
- **SASRec** (Kang and McAuley, 2018) models user behavior sequences based on the Transformer self-attention mechanism to provide efficient sequence recommendations.

A.3 Baseline Attack Methods

Matrix-Poisoning is a gray-box attack against recommender systems. It does not need to access recommender systems' internal data. Therefore, we compared Matrix-Poison with traditional attack methods and existing low-knowledge attack baseline methods, and we referred to these baseline methods for the highest score assigned to the item goal:

- **Random Attack:** (Lam and Riedl, 2004) By injecting fake users to score random items randomly and assigning high/low scores to target items, the recommender system is manipulated in a simple but less covert way.

- **Average Attack:** (Lam and Riedl, 2004) The recommendation system poisoning attack with higher concealment and effectiveness is realized by scoring the target items based on the average score of the items by fake users and assigning high/low scores to the target items.
- **Bandwagon Attack:** (Burke et al., 2006) By assigning high scores (or low scores) to popular items and target items, and using the conformity effect to construct fake user ratings, the recommendation system poisoning attack with high concealment is realized with low knowledge requirements.
- **Segmented Attack:**(Burke et al., 2005) By selecting items related to the target item for specific user groups and giving them high scores (or low scores), the fake user scores are constructed to improve the attack effect and concealment.

A.4 Evaluation Details

We evaluate attack effectiveness using Hit@k, NDCG@k, and Prediction Shift. For category-level poisoning, the target set is $\mathcal{T} = C_t$; for item-level evaluation, it reduces to $\mathcal{T} = \{i_t\}$. Let $\mathcal{R}_u^k$ be the top-k recommendation list of user u and $\mathcal{U}_e$ be the evaluation user set. Hit@k is computed as

$$\text{Hit@}k = \frac{1}{|\mathcal{U}_e|} \sum_{u \in \mathcal{U}_e} \mathbf{1}\left[\mathcal{R}_u^k \cap \mathcal{T} \neq \emptyset\right]. \tag{22}$$

NDCG@k is computed by assigning relevance 1 to items in $\mathcal{T}$ and 0 otherwise:

$$\text{NDCG@}k = \frac{1}{|\mathcal{U}_e|} \sum_{u \in \mathcal{U}_e} \frac{\text{DCG}_u@k}{\text{IDCG}_u@k}, \tag{23}$$

where

$$\text{DCG}_u@k = \sum_{j=1}^{k} \frac{\mathbf{1}[\mathcal{R}_{u,j} \in \mathcal{T}]}{\log_2(j+1)}. \tag{24}$$

Here, $\mathcal{R}_{u,j}$ denotes the item ranked at position j. If no target item appears in the ideal top-k list, the corresponding NDCG value is set to zero. Prediction Shift is defined as the average change in the predicted score of the target item:

$$\delta_p = \frac{1}{|\mathcal{U}_e|} \sum_{u \in \mathcal{U}_e} \left[f_{\text{poison}}(u, i_t) - f_{\text{clean}}(u, i_t)\right]. \tag{25}$$

For Nuke attacks, lower Hit@k, NDCG@k, and more negative Prediction Shift indicate stronger suppression. For Push attacks, higher Hit@k, NDCG@k, and positive Prediction Shift indicate stronger promotion.

A.5 Baseline Adaptation and Budget Control

AUSH, GOAT, and PC-Attack are originally designed for item-level poisoning. To adapt them to category-level evaluation, we use the same target item $i_t \in C_t$ selected for Matrix-Poison as the explicit attack target, while evaluating their impact on the category-level target set $\mathcal{T} = C_t$. This preserves the original attack interface of these baselines and makes the evaluation consistent with our category-level objective.

All methods follow the same injection budget. Given poisoning ratio ϵ, the number of fake users is fixed as $|\mathcal{U}_M| = \text{round}(|\mathcal{U}|\epsilon)$. The maximum malicious profile length is matched across methods whenever the baseline exposes this parameter. For filler-item construction, target-category items are excluded from the filler pool unless required by the original baseline design, and the same candidate pool is used across methods. For PC-Attack, the number of popularity-based filler items is fixed across datasets. Thus, the comparison controls fake-user budget, profile size, target item, victim model, data split, and evaluation metric.

A.6 Matrix-Poison Self-Evolving Optimization Pseudocode

Algorithm 1 Matrix-Poison Self-Evolving Optimization

Require: Clean dataset D, target category C_t, target item i_t, poisoning budget ϵ, surrogate model f_{shadow}

Initialize analysis memory $\mathcal{M}_a$ and planning memory $\mathcal{M}_p$
Compute category statistics and initialize primitive mixture $\{\beta_k\}$
for $t = 1$ to T **do**
  Analyze category vulnerability and update $\mathcal{M}_a$
  Generate candidate poisoning policy π_t using the planning agent
  Construct malicious profiles $D_M^{(t)}$ under budget ϵ
  Train or update f_{shadow} on $D \cup D_M^{(t)}$
  Evaluate attack effect and stealthiness
  Parse semantic feedback $\nabla_{\text{sem}}^{(t)} = \langle d, m, I \rangle$
  Update policy using fine-tuning, switching, or hybridization operators
end for
return Best poisoning policy according to attack-stealth trade-off

A.7 Threat Model&Capabilities Table

To make the attack assumptions explicit, we summarize the threat model and attacker capabilities in Table 6. Matrix-Poison is evaluated under a gray-box poisoning setting: the attacker cannot access the victim recommender's parameters, gradients, or training pipeline, but can construct a shadow dataset and train surrogate recommenders to guide attack planning. This setting reflects a realistic scenario in which item metadata and limited recommendation feedback are publicly observable, while internal model details remain unavailable. The attacker is constrained by a fixed poisoning budget and can only inject synthetic interactions through controlled accounts; modifying benign users or tampering with the deployed recommender is excluded.

Aspect	Assumption in Matrix-Poison
Victim model access	No access to model parameters, gradients, or training pipeline.
Training data access	No direct access to the full victim training set; the attacker can collect or sample a shadow dataset.
Item metadata	The attacker can observe public item metadata such as title, category, and description.
Category knowledge	The attacker knows the target category label C_t and selects a target item $i_t \in C_t$.
Injection capability	The attacker can inject at most ϵ fraction of malicious interactions through controlled accounts.
Feedback signal	The attacker observes limited ranking outputs or surrogate validation metrics, but not internal scores.
Cross-market setting	The attacker evolves the strategy on a source market and transfers it to a target market without retraining.
Excluded capabilities	The attacker cannot modify benign users, delete interactions, or tamper with model code.

Table 6: Threat model and attacker capabilities considered in this work.

A.8 Adaptation of Item-level Baselines to Category-level Attacks

AUSH, GOAT, and PC-Attack are originally designed for item-level poisoning, whereas our evaluation focuses on category-level manipulation. To ensure a fair comparison, we adapt these baselines to the category-level setting through a target-expansion protocol. Specifically, for each target category C_t, we first select the same target item $i_t \in C_t$ used by Matrix-Poison according to the predefined target-selection rule. The item-level baseline is then executed with i_t as its explicit attack target, while its effect is evaluated not only on i_t but also on the category-level ranking metrics over C_t. This keeps the attack interface consistent with the original baseline design while aligning the evaluation with our category-level objective.

For budget control, all baselines use the same poisoning ratio ϵ, i.e., the number of injected fake users is fixed as $|\mathcal{U}_M| = \mathrm{round}(|\mathcal{U}|\epsilon)$. The maximum profile length and filler-item size are also matched to Matrix-Poison whenever the baseline exposes such parameters. When a baseline requires item-level filler sampling, we draw filler items from the same candidate pool used in our experiments and exclude target-category items unless the original method explicitly requires them. This prevents a baseline from obtaining additional category exposure through a larger or easier poisoning budget.

For PC-Attack, which depends on popularity-based filler construction, we keep the number of popular filler items fixed across datasets and report the value used in the implementation. For AUSH and GOAT, we retain their original profile-generation logic but constrain the generated malicious profiles to the same fake-user budget and profile-length range as Matrix-Poison. In all cases, the poisoned training set is formed as $\mathcal{D}' = \mathcal{D} \cup \mathcal{D}_M$, and the same victim model, train/test split, evaluation users, and metrics are used. Therefore, the comparison measures how well item-level attack mechanisms transfer to category-level objectives under matched injection budgets and profile-size constraints.

A.9 Prompt Templates and Agent Trace

For safety reasons, we report redacted prompt templates rather than full operational prompts or complete target-specific trajectories. This appendix is intended to improve reproducibility at the methodological level while avoiding the release of directly executable misuse artifacts. Matrix-Poison uses a role-based agent coordinator with four functional agents: Analysis Agent, Planning Agent, Judgement Agent, and Self-Evolving Optimizer Agent. Each agent is instructed to return structured JSON, and invalid or unparsable responses are replaced by deterministic fallback rules. This design makes the optimization process auditable while reducing sensitivity to uncontrolled natural-language outputs.

Semantic Gradient Representation. In the main text, the semantic gradient is defined as

$$\nabla_{\text{sem}} = \text{LLMAnalysis}(D, e_t, O_{\text{adv}}), \tag{26}$$

where D denotes the observed data context, e_t denotes the current attack feedback at iteration t, and O_{adv} denotes the adversarial objective. Operationally, Matrix-Poison parses this feedback into a structured tuple:

$$\nabla_{\text{sem}} = \langle d_t, m_t, I_t \rangle, \tag{27}$$

where $d_t \in \{+1, -1\}$ represents the adjustment direction, m_t represents the adjustment magnitude, and I_t denotes the concrete instruction used to refine the poisoning strategy. In the implementation, this tuple is instantiated through JSON-controlled agent outputs, including ranked attack-method preferences, updated mixture weights, judgement decisions, and adjusted poisoning budgets.

Agent Memory. Matrix-Poison stores the optimization trajectory as iteration-level memory. At each iteration t, the memory record is defined as:

$$\begin{aligned} M_t = \big\{ & t, \epsilon_t, w_t, O_t, \delta_{p,t}, \text{Hit@}k_t, \text{NDCG@}k_t, \\ & S_t, R_t, \Delta_{\text{clean},t}, J_t \big\}. \end{aligned} \tag{28}$$

where ϵ_t is the poisoning budget, w_t is the attack-mixture weight vector, O_t is the attack objective, $\delta_{p,t}$ is the prediction shift, S_t is the stealth penalty, R_t is the final reward, $\Delta_{\text{clean},t}$ is the clean-utility drop, and J_t indicates whether the candidate passes the judgement phase.

The trace in Table 8 illustrates how Matrix-Poison records the optimization process. The first iteration identifies a candidate strategy with a negative prediction shift and low target exposure. The self-evolving mechanism then adjusts the poisoning budget for the next iteration while preserving the same mixture weights, since the candidate already satisfies the judgment constraints. This memory structure enables later analysis of why a particular poisoning policy was selected and how the agent balances attack effectiveness with stealthiness.

A.10 Atomic Attack Basis

The unified poisoning generator $\mathcal{M}(\boldsymbol{\theta}_t; \mathcal{D}, \mathcal{C}_t)$ is built on a fixed atomic attack basis. In our implementation, $K = 4$ and the basis consists of four canonical low-knowledge poisoning operators:

$$B = \{\textit{Random}, \textit{Average}, \textit{Bandwagon}, \textit{Segmented}\}. \tag{29}$$

The mixture vector $\mathbf{w}_t = (w_{t,1}, \ldots, w_{t,4}) \in \Delta^3$ determines the fraction of malicious users generated by each operator. Given the poisoning ratio ϵ_t, the total number of malicious users is

$$|\mathcal{U}_M^{(t)}| = \text{round}(|\mathcal{U}|\epsilon_t), \tag{30}$$

and each basis operator receives

$$n_{t,k} = \text{round}(w_{t,k}|\mathcal{U}_M^{(t)}|), \qquad k = 1, \ldots, 4, \tag{31}$$

with a final correction to ensure $\sum_k n_{t,k} = |\mathcal{U}_M^{(t)}|$.

The four operators differ only in how filler items and ratings are generated. *Random* samples filler items uniformly from the non-target pool. *Average* assigns filler ratings close to item-level empirical means. *Bandwagon* anchors fake profiles on popular non-target items. *Segmented* samples fillers from items related to the target user segment or neighboring category context. For all operators, target-category feedback is assigned according to the attack intent: positive feedback for `Push` and suppressive or redirected feedback for `Nuke`. The auxiliary parameters $\boldsymbol{\eta}_t$ control shared implementation choices such as profile length, filler pool size, popularity-anchor size, and target-feedback value.

Thus, Matrix-Poison does not rely on a new primitive poisoning rule. Its core mechanism is the adaptive composition of these fixed operators through $\mathbf{w}_t$, together with self-evolving updates of ϵ_t and $\boldsymbol{\eta}_t$.

A.11 Details of Cross-market Setting

We investigate whether Matrix-Poison learns transferable attack knowledge rather than overfitting to a single domain. As shown in Fig. 4, the protocol follows three stages: (i) in the source market, we run clean RS training and self-evolving attack search to obtain a strategy package (attack-weight prior, epsilon schedule, memory summary, and judge rules); (ii) we transfer only policy/memory-level knowledge, without transferring raw interaction data or user/item identities; and (iii) in the target market, we initialize the planner with the transferred strategy and evaluate attack effectiveness using Hit@10, NDCG@10, prediction shift, and stealth/detection metrics. We also include an optional feedback loop for a few-step adaptation, enabling a direct comparison between zero-shot and few-shot transfer.

A.12 Update Rules and Hyperparameters

All agent outputs are constrained to strict JSON schemas. If the Analysis Agent returns an invalid

Agent	Input Context	Redacted Prompt / Output Schema
Analysis Agent	Attack intent, target category, and method sensitivity scores obtained from probe attacks.	`Rank attack methods by expected category-level vulnerability. intent=[INTENT], target_category=[CATEGORY], method_sensitivity={...}. Return JSON ONLY: {"ranking": ["m1","m2","m3","m4"], "reason": "short"}.`
Planning Agent	Base attack plan, current mixture weights, iteration index, stagnation counter, and attack intent.	`Optimize mixture weights for attack methods. intent=[INTENT], iteration=[t], stagnation_steps=[s]. base_plan={...}, current_weights={...}. Return JSON ONLY: {"weights": {"random": x1, "average": x2, "bandwagon": x3, "segmented": x4}, "reason": "short"}.`
Judgement Agent	Candidate metrics, attack direction, clean-utility tolerance, and minimal target-shift constraint.	`Assess whether the candidate's attack should pass judging. is_push=[BOOL], clean_drop_tolerance=[VALUE], push_min_shift=[VALUE]. metrics={...}. Return JSON ONLY: {"pass_judge": true, "reward_adjust": 0.0, "reason": "short"}.`
Self-Evolving Optimizer	Current poisoning budget, objective value, stealth penalty, prediction shift, and budget bounds.	`Update epsilon for next iteration. current_epsilon=[VALUE], objective=[VALUE], stealth_penalty=[VALUE]. prediction_shift=[VALUE], epsilon_min=[VALUE], epsilon_max=[VALUE]. Return JSON ONLY: {"next_epsilon": [VALUE], "reason": "short"}.`

Table 7: Redacted prompt templates used by the Matrix-Poison agent coordinator. Sensitive target-specific details are omitted, while the input-output structure is preserved for reproducibility.

Iter.	ϵ_t	Weights	δ_p	Hit@10	NDCG@10	Pass
1	0.0050	$(0.12, 0.24, 0.48, 0.16)$	-2.17×10^{-4}	0.0010	0.0003	True
2	0.0078	$(0.12, 0.24, 0.48, 0.16)$	-8.22×10^{-5}	0.0010	0.0005	True

Table 8: An anonymized example of the Matrix-Poison iteration trace. The weight tuple follows the order of Random, Average, Bandwagon, and Segmented attack primitives.

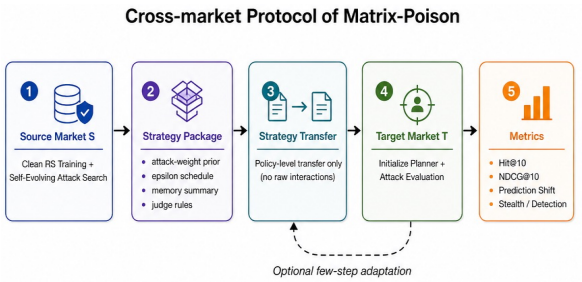

Figure 4: Cross-market protocol of Matrix-Poison. We evolve attack strategies in a source market, transfer policy-level knowledge (instead of raw interaction data), and evaluate effectiveness in a target market under zero-shot/few-step adaptation.

Parameter	Default / Rule
ϵ_0	0.005
$\epsilon_{\min}$, $\epsilon_{\max}^{\text{nuke}}$	0.001, 0.2
$\epsilon_{\max}^{\text{push}}$	model-capped, upper 0.5
α_{probe}	0.4
τ_p (push min shift)	1×10^{-4}
τ_c (clean-drop tolerance)	0.06
λ_s	0.25
γ (push interaction penalty)	6.0
η_1, η_2, η_3	3.0, 3.0, 1.0
λ_m	0.10 (first iter), 0.15 (later)
(w_s, w_r) push (BPR/NeuMF)	(0.70, 1.10)
(w_s, w_r) push (LightGCN)	(0.65, 1.15)
(w_s, w_r) push (SASRec)	(0.60, 1.20)
(w_s, w_r) nuke (all models)	(0.20, 1.00)

Table 9: Implementation-level hyperparameters for reproducibility.

ranking, candidates are sorted by measured sensitivity scores. If the Planning Agent returns invalid weights, the system uses the rank-based default weights and projects them to the simplex. If the Self-Evolving Optimizer returns an invalid poisoning ratio, ϵ_{t+1} is computed by the deterministic multiplier rule and clipped to $[\epsilon_{\min}, \epsilon_{\max}^{\text{mode}}]$. A candidate is feasible only if it satisfies the minimum target-shift requirement, the clean-utility tolerance, the maximum interaction-ratio bound, and the maximum stealth penalty. Table 9 reports the values used in all main experiments. Table 9 specifies all key constants used in the implementation.

B Further Discussion

B.1 Highlights & Potential Defense Methods

What critical vulnerabilities of recommender systems are exposed by category-level and cross-market poisoning attacks by Matrix-Poison? To answer this question, we summarize the key findings and defense implications as follows:

- **Superior Attack Effectiveness:** Matrix-Poison consistently outperforms state-of-the-art baselines in both single-market push and nuke settings, showing strong capability to manipulate category-level recommendation outcomes.
- **Robust Cross-Market Generalization:** Agents evolved in a dominant market transfer effectively to other markets with different user behaviors, without retraining.
- **High Stealthiness:** The attack remains largely imperceptible by preserving rating-distribution statistics and minimizing utility degradation for benign users.
- **Anomaly Detection:** Statistical screening and graph-based clustering can identify coordinated abnormal behaviors and reduce malicious interaction influence.
- **Model Robustness Enhancement:** Regularization and adversarial training improve stability against contaminated interactions and reduce sensitivity to poisoning noise.
- **Data Preprocessing:** Pre-training filtering (e.g., threshold checks and behavior verification) helps block suspicious interactions before they propagate into model learning.
- **Integrated and Future-Proof Defense:** A unified pipeline combining detection, prevention, and robustness can balance protection and computational cost; privacy-preserving paradigms (e.g., federated learning and differential privacy) are promising directions for defending against evolving category-level attacks.

C Additional Experiments

C.1 Experiments on real contributions of LLM agents

To isolate the contribution of the LLM-driven planner, we compare Matrix-Poison against several non-LLM optimization strategies with the same poisoning and validation budgets. As shown in Table 10, the LLM planner achieves stronger attack effectiveness with fewer search trials, suggesting that semantic feedback and memory-guided refinement provide benefits beyond simple hyperparameter tuning.

C.2 Poisoning budget sensitivity

We further evaluate Matrix-Poison under different poisoning budgets. Figure 5 shows that the attack effect increases as ϵ grows, while Matrix-Poison remains effective even under small budgets. This indicates that the proposed self-evolving planner can exploit category-level vulnerabilities without relying on excessive fake interactions.

C.3 Computational Cost Profiling

We further profile the computational overhead of Matrix-Poison to clarify its practical cost. The profiling is conducted on the Automotive dataset with four victim recommenders under the Nuke setting, using the same search configuration as the main implementation: $T = 5$ self-evolving iterations, 20 clean-training epochs, 4 probe epochs, and 8 judge epochs. All runs are executed on a server with 16 CPU cores, 80GB system memory, and one NVIDIA RTX 4090 D GPU. We disable external LLM API calls in this profiling experiment and use the deterministic JSON fallback, so the reported wall-clock time mainly reflects local surrogate training, probe evaluation, and candidate judging. External LLM latency is therefore isolated from the recommender-side cost.

For each attack search, Matrix-Poison trains one clean recommender, retrains $K = 4$ probe recommenders in the analysis phase, and retrains one judged candidate per self-evolving iteration. Thus, a run with $T = 5$ performs $1 + 4 + 5 = 10$ local model-training calls. As shown in Table 12, the average wall-clock time is 20.07 seconds per attack search on Automotive, with SASRec being the most expensive model. The peak allocated GPU memory remains below 2.2GB in all profiled runs.

Planner	Hit@10↓	P. Shift↓	Queries / Trials
Rule-based heuristic	0.0100	-0.0001	5
Random search	0.0000	-0.0002	8
Grid search	0.0000	-0.0002	11
Bayesian optimization	0.0000	-0.0002	8
Evolutionary search (no LLM)	0.0000	-0.0001	12
LLM planner w/o memory	0.0010	-0.0001	1
Matrix-Poison w/ GPT-4o	0.0100	-0.0001	5
Matrix-Poison w/ DeepSeek-R1	0.0100	-0.0001	5

Table 10: Comparison of different planning strategies.

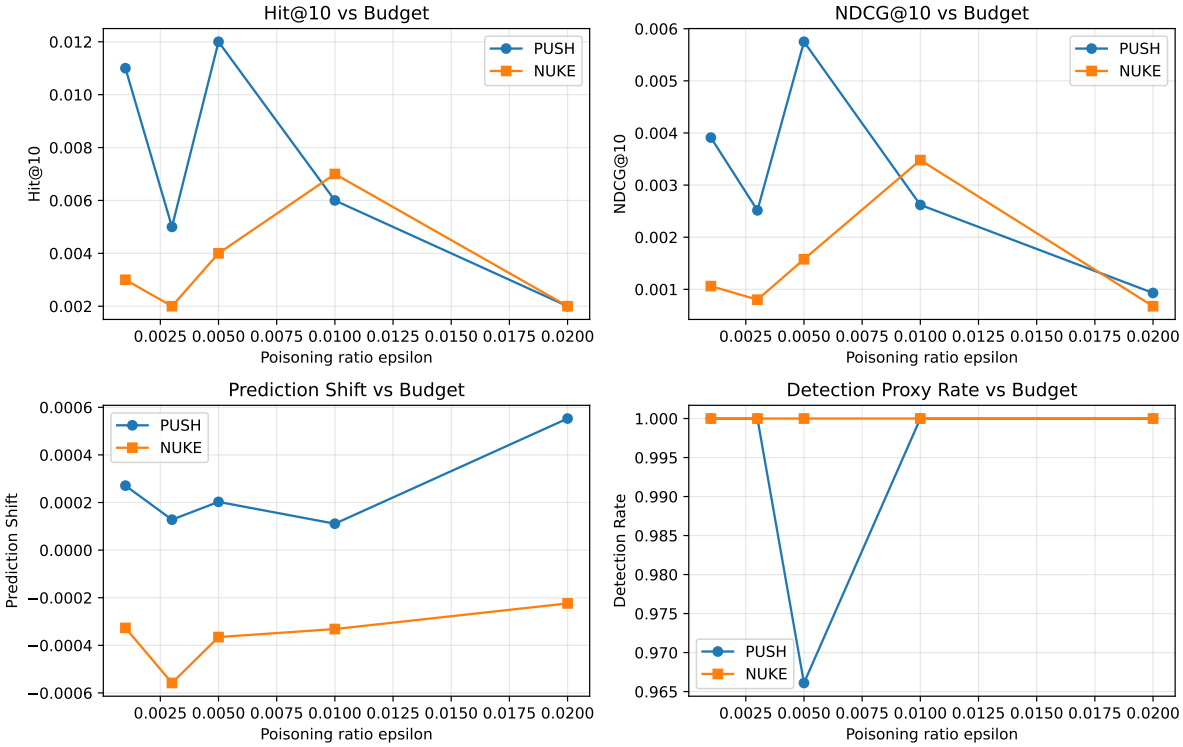

Figure 5: Attack effectiveness under different poisoning budgets $\epsilon \in \{0.001, 0.003, 0.005, 0.01, 0.02\}$. Curves report Hit@10, NDCG@10, prediction shift, and detection proxy rate for Push/Nuke settings.

Model	Iter.	Train calls	Time (s)	Peak GPU (MB)
BPR	5	10	14.69	20.16
NeuMF	5	10	15.49	84.86
LightGCN	5	10	15.31	43.72
SASRec	5	10	34.78	2160.46
Average	5.00	10.00	20.07	577.30

Table 12: Computational cost profiling of Matrix-Poison on Automotive. "Train calls" counts one clean training run, four probe retraining runs, and one judge retraining run per self-evolving iteration.